Compressing ACAS-Xu LUTs with Binary Decision Diagrams

Journées FAC 2026

Martin BONIOL

Supervisors : X. Thirioux - J. Brunel - C. Garion - J-B. Chaudron

Context & Introduction

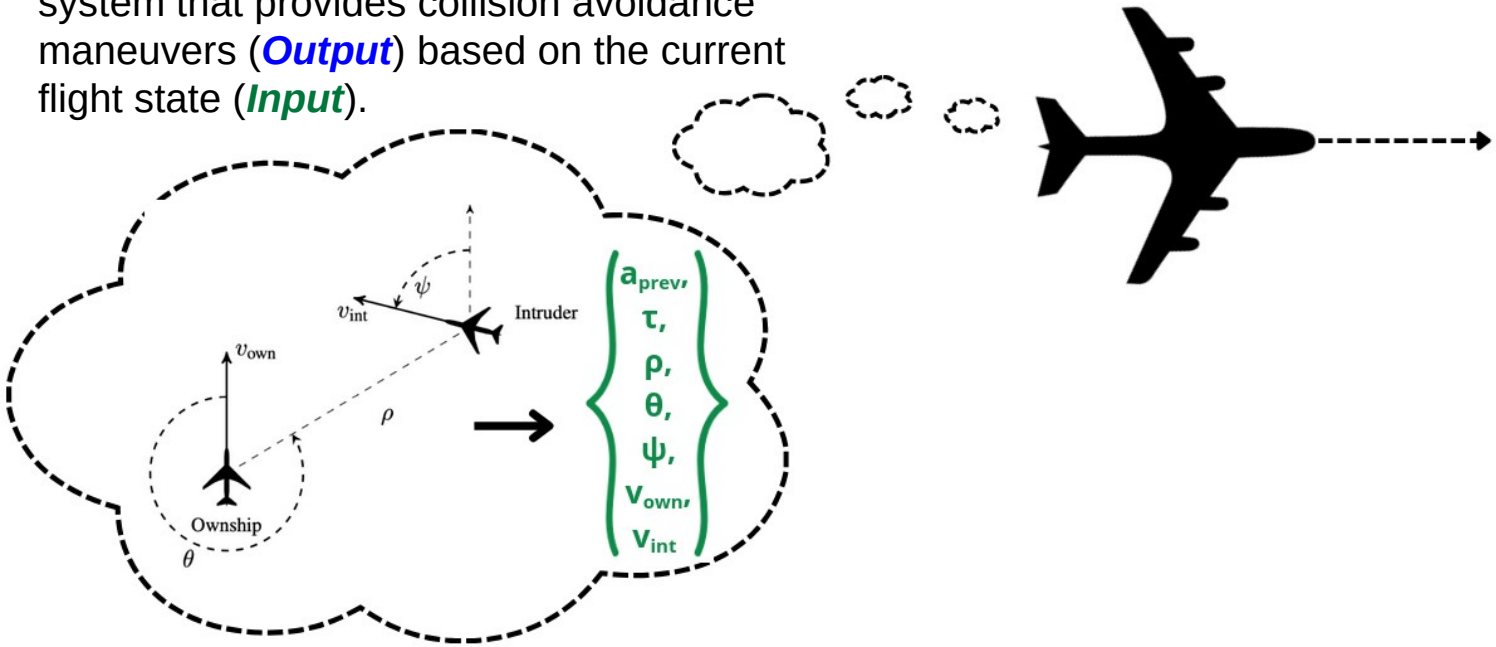
Anti-collision systems for air traffic are complex but essential

- TCAS since the 80s.

- ACAS-X new generation :
  - ACAS-Xa
  - ACAS-Xo
  - ACAS-Xu

Context & Introduction

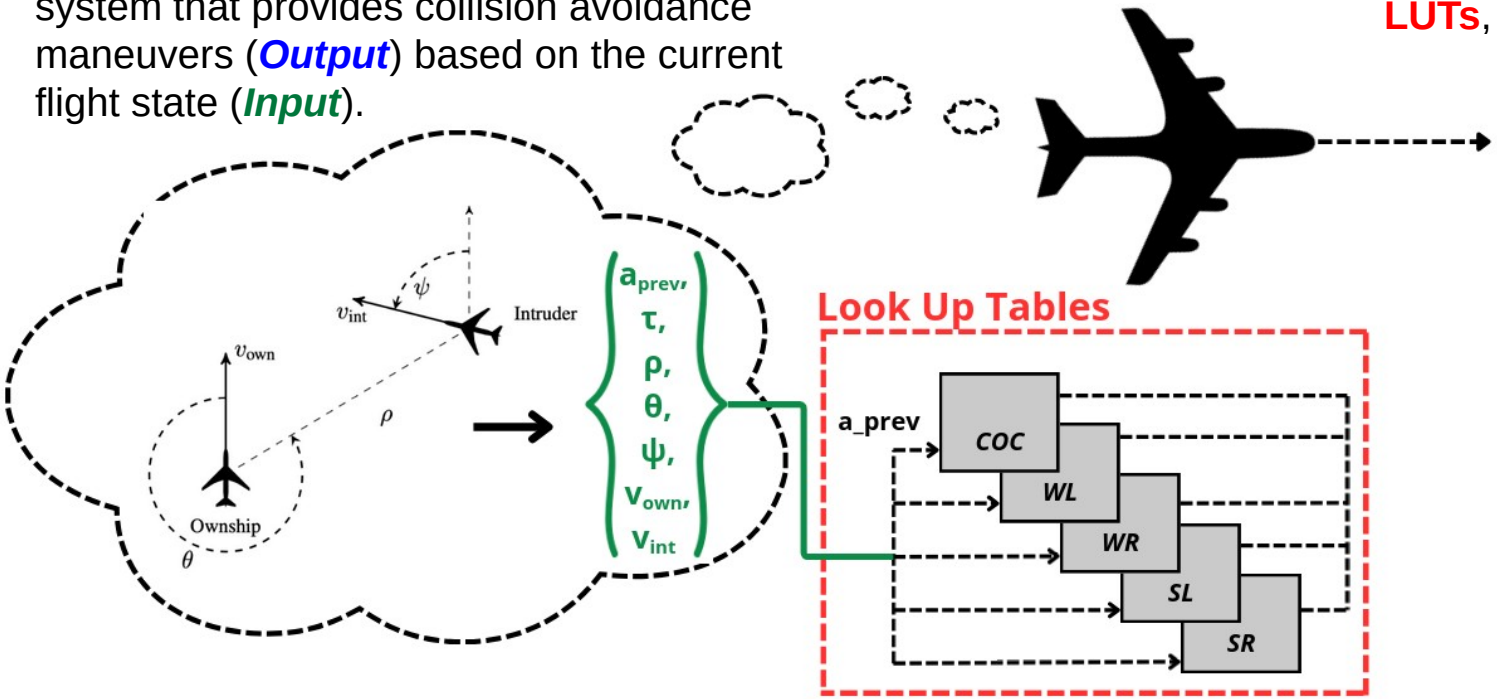
ACAS-Xu : An airborne collision avoidance system that provides collision avoidance maneuvers (***Output***) based on the current flight state (***Input***).



Context & Introduction

ACAS-Xu : An airborne collision avoidance system that provides collision avoidance maneuvers (***Output***) based on the current flight state (***Input***).

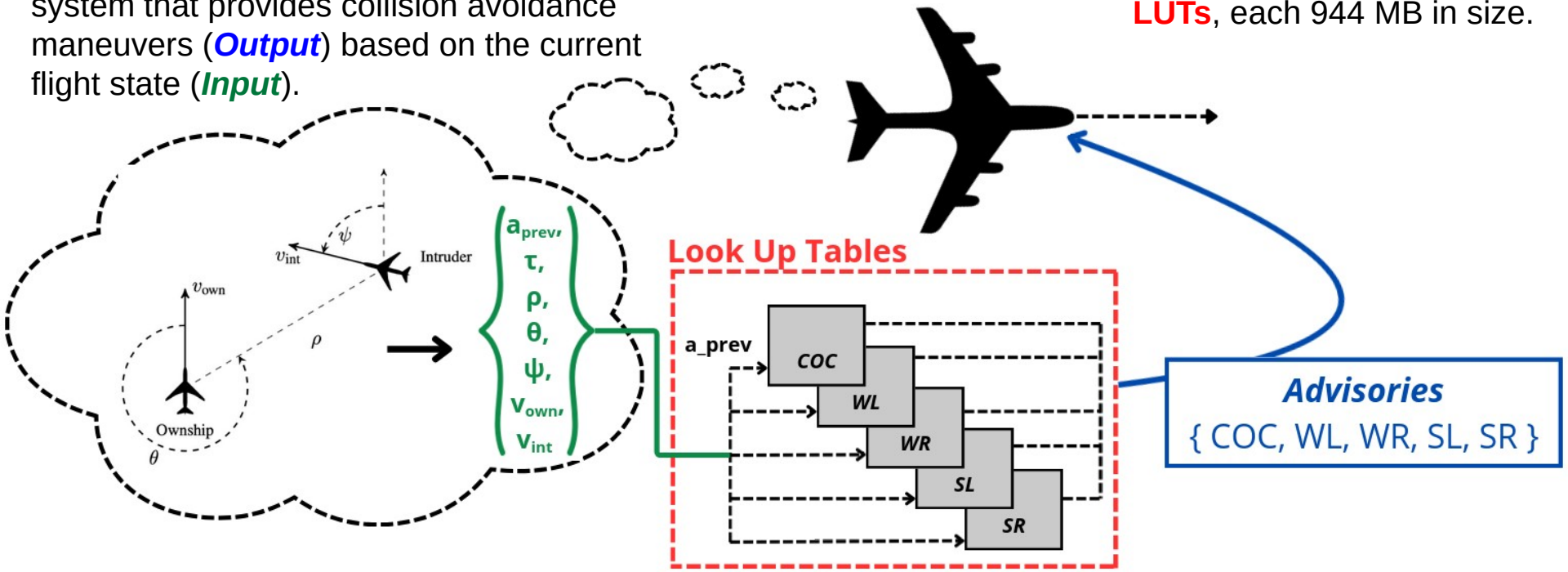
Operates using 5 **LARGE LUTs**, each 944 MB in size.



Context & Introduction

ACAS-Xu : An airborne collision avoidance system that provides collision avoidance maneuvers (***Output***) based on the current flight state (***Input***).

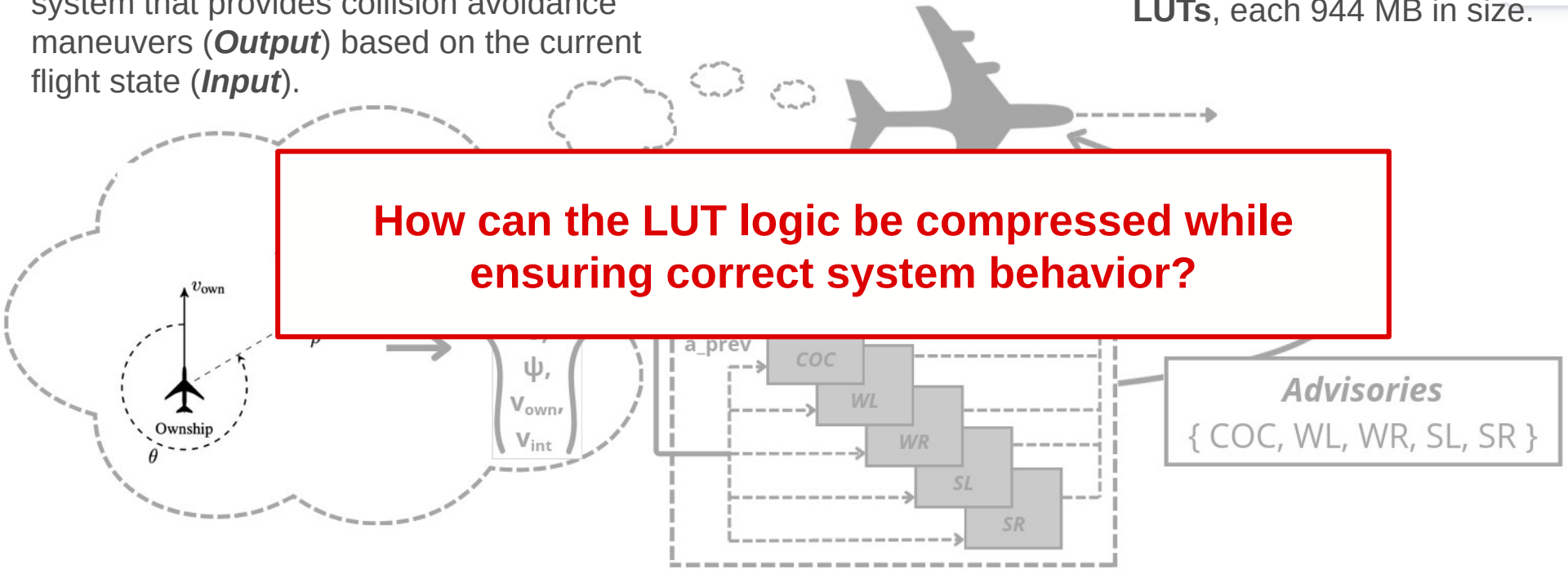
Operates using 5 **LARGE LUTs**, each 944 MB in size.



Context & Introduction

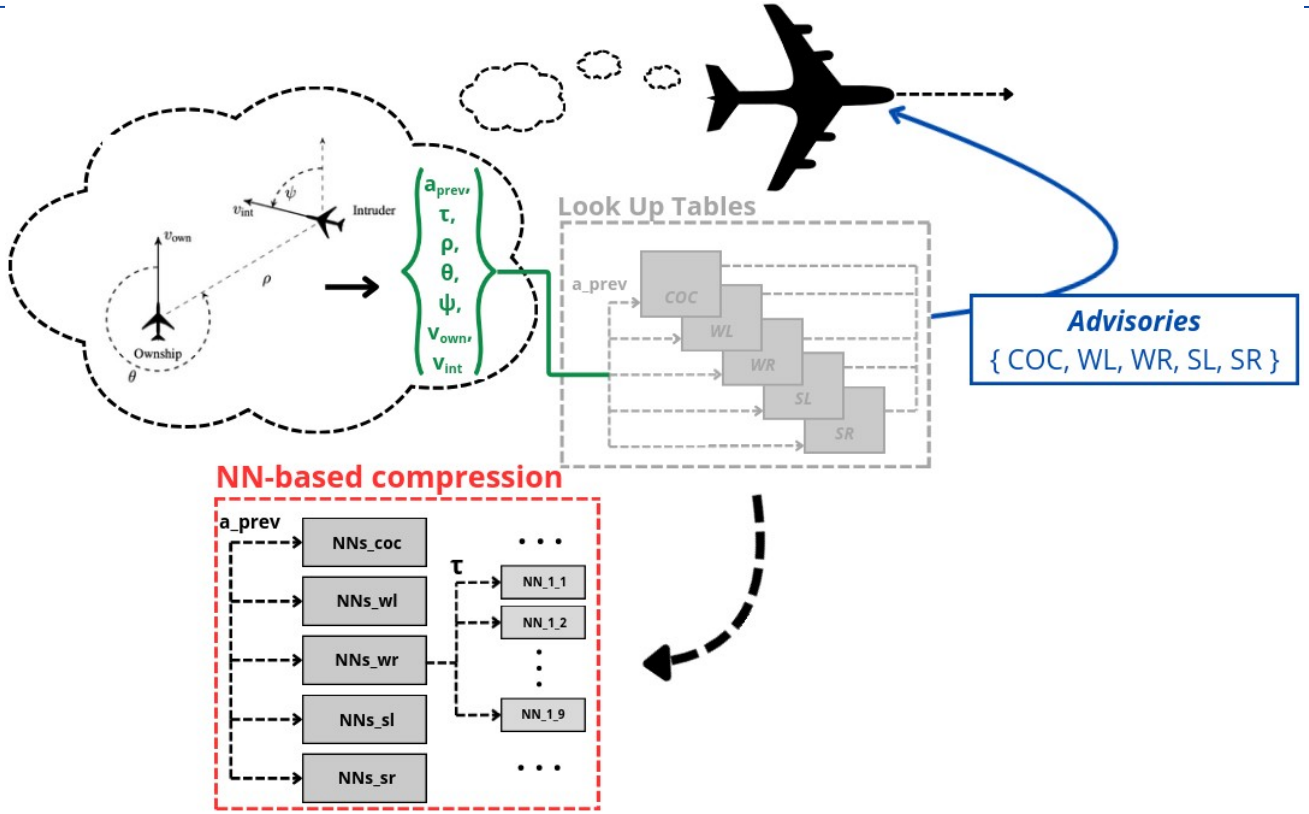
ACAS-Xu : An airborne collision avoidance system that provides collision avoidance maneuvers (***Output***) based on the current flight state (***Input***).

Operates using 5 **LARGE LUTs**, each 944 MB in size.

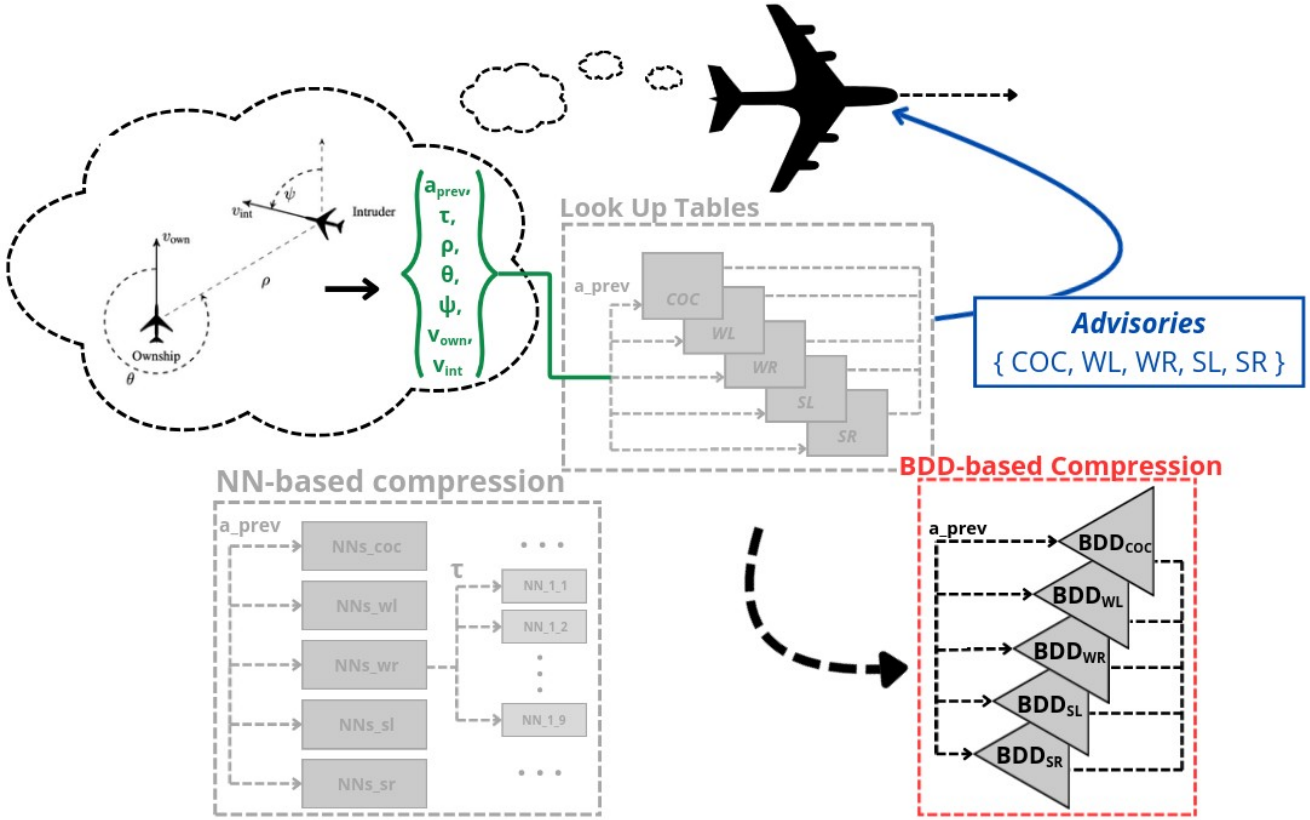


How can the LUT logic be compressed while ensuring correct system behavior?

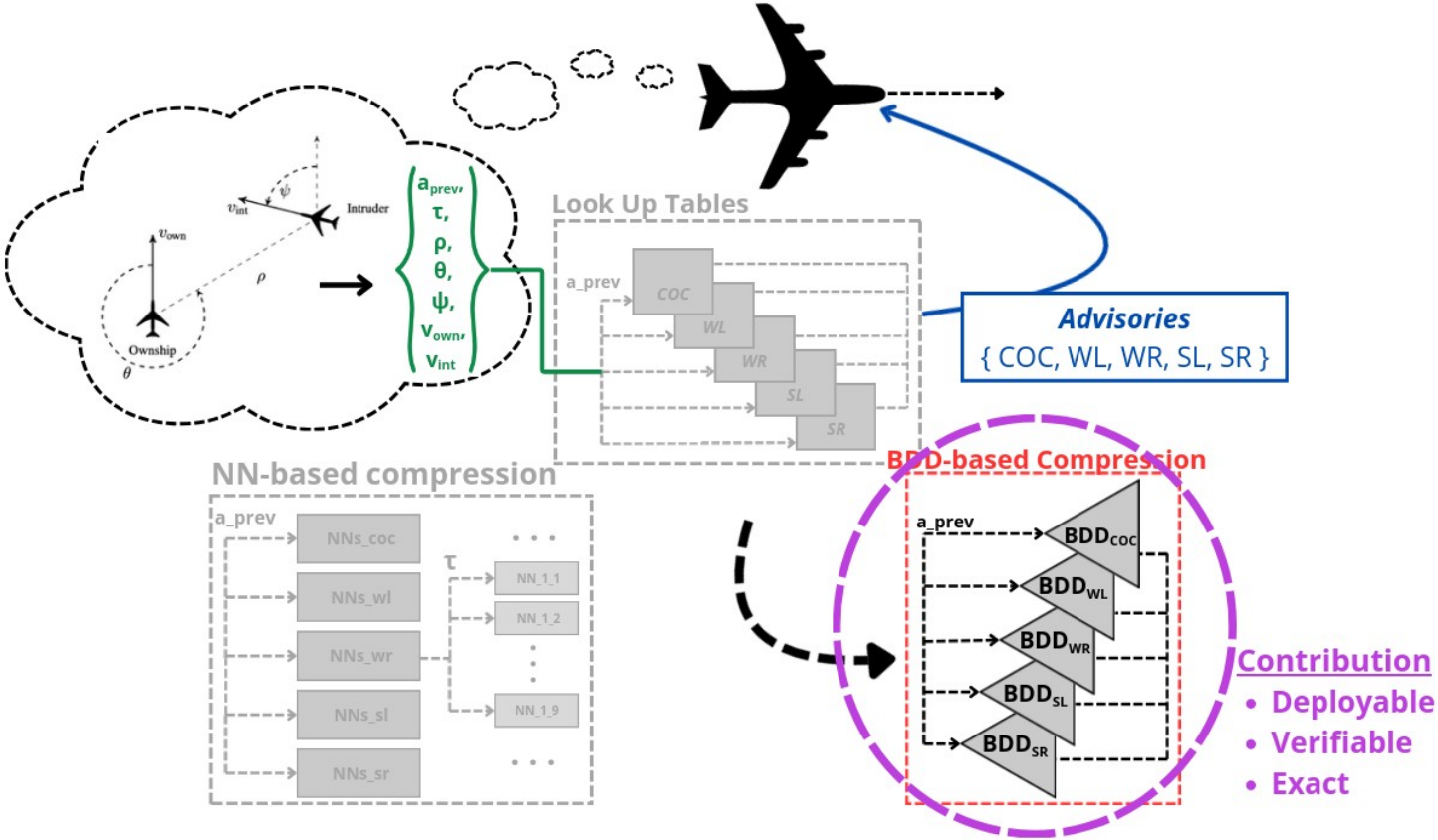
Context & Introduction



Context & Introduction

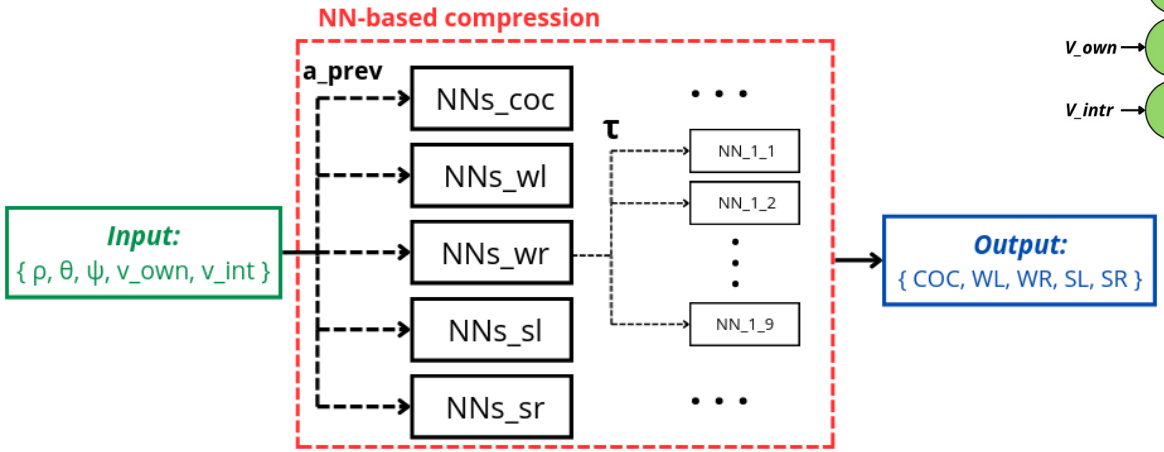


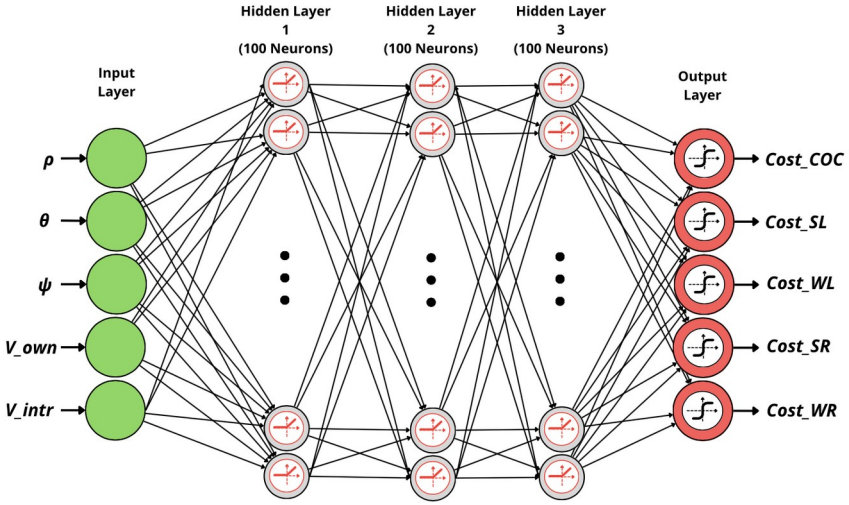
Context & Introduction



LUTs compression with Neural Network

- **NNs proposed in the literature:**
  - Julian (45 NNs) – ***J_nn***





- Fully connected, with ReLU.
- 5 inputs : 5 states parameters (discretization on τ).
- 5 outputs : costs of the next actions.

LUTs compression with Neural Network

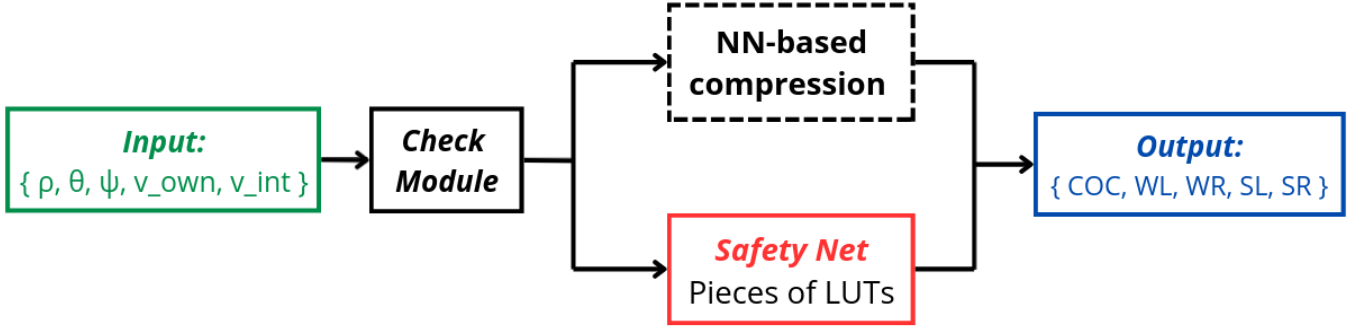
- **NNs proposed in the literature:**
  - Julian (45 NNs) – ***J_nn***

NN	LUT Points (COC)	LUT Points (Non-COC)
NN_1_1	97.76%	57.44%
NN_2_1	96.58%	69.95%
NN_3_1	92.30%	68.53%
NN_4_1	98.54%	59.58%
NN_5_1	98.22%	59.87%

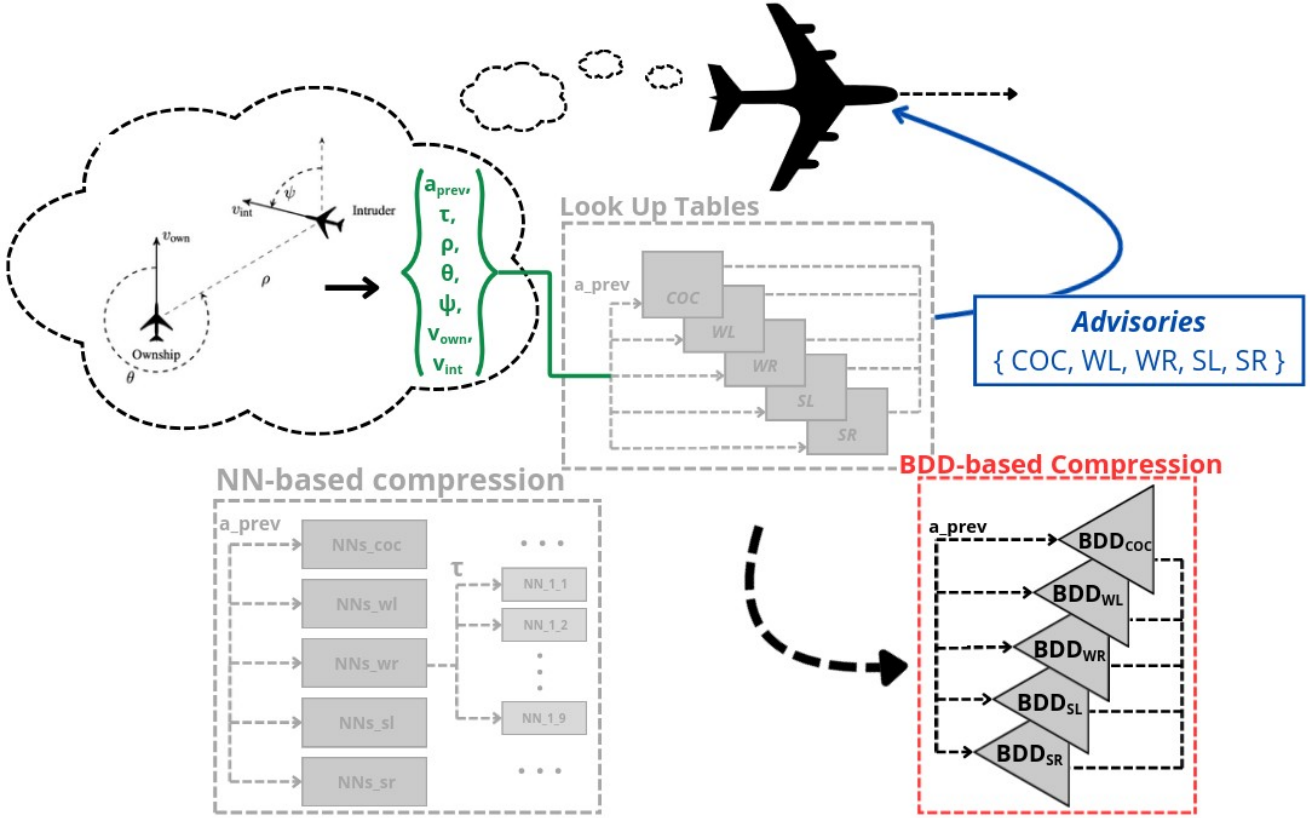
LUTs compression with Neural Network

- **NNs proposed in the literature:**
  - Julian (45 NNs) – ***J_nn***

NN	LUT Points (COC)	LUT Points (Non-COC)
NN_1_1	97.76%	57.44%
NN_2_1	96.58%	69.95%
NN_3_1	92.30%	68.53%
NN_4_1	98.54%	59.58%
NN_5_1	98.22%	59.87%



Compression via Binary Decision Diagrams

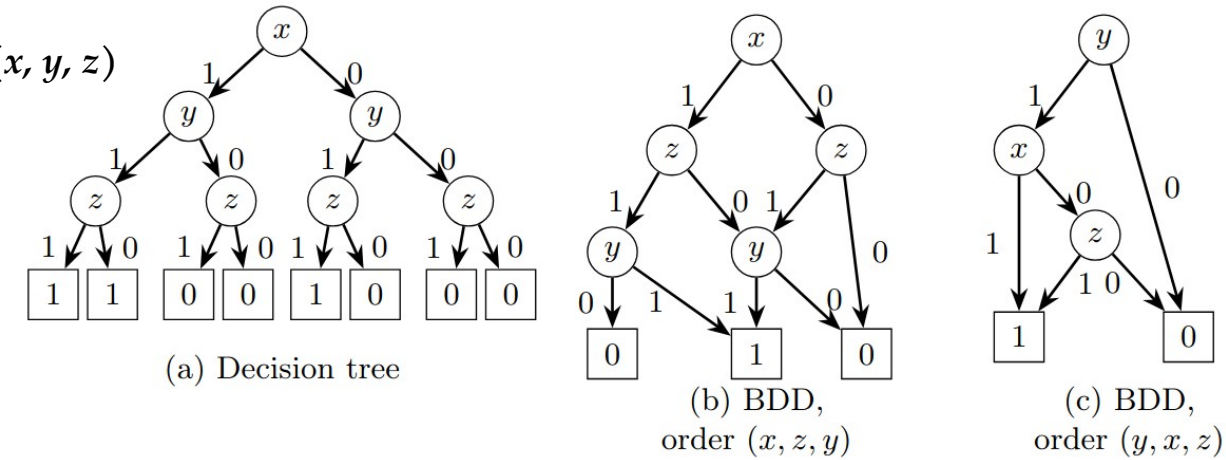


What is a Binary Decision Diagram (BDD)

- **A BDD is a compact graph representation of a Boolean function, where identical decision substructures are shared.**
  - Internal nodes = Boolean variables
  - Edges = variable outcomes (true / false)
  - Leaves = output values (0 or 1)

- **Example : Representation of $f(x, y, z)$**

  $f(x, y, z) = (x \wedge y) \vee (y \wedge z)$



Compression via BDDs

- **Encoding LUT data into BDDs:**
  - For 1 LUT : 5 BDDs, one per advisory.
  - Each advisory is encoded as a separate BDD representing the set of states where it is selected

State Tuple

$$\left\{ \begin{array}{c} \tau, \\ \rho, \\ \theta, \\ \psi, \\ v_{own}, \\ v_{int} \end{array} \right\}$$

$\xrightarrow{\text{Mapping along each dimension}}$

Indices Tuple

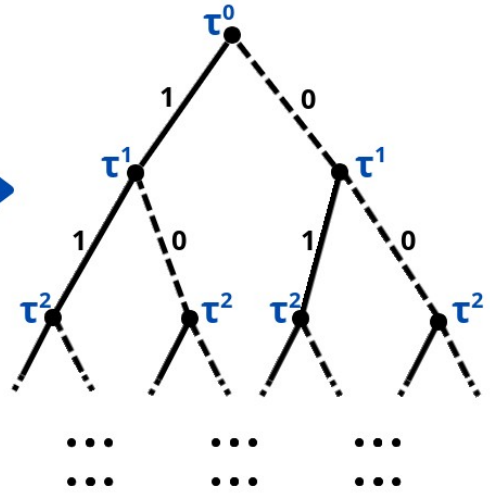
$$\left\{ \begin{array}{c} \tau^i \in [0, 9], \\ \rho^i \in [0, 40], \\ \theta^i \in [0, 38], \\ \psi^i \in [0, 38], \\ v^i_{own} \in [0, 11], \\ v^i_{int} \in [0, 11] \end{array} \right\}$$

$\xrightarrow{\text{Gray Code}}$

Boolean Variables

$$\left\{ \begin{array}{c} \tau^0\ \tau^1\ \tau^2\ \tau^3, \\ \rho^0\ \rho^1\ \rho^2\ \rho^3\ \rho^4\ \rho^5, \\ \theta^0\ \theta^1\ \theta^2\ \theta^3\ \theta^4\ \theta^5, \\ \psi^0\ \psi^1\ \psi^2\ \psi^3\ \psi^4\ \psi^5, \\ v^0_{own}\ v^1_{own}\ v^2_{own}\ v^3_{own}, \\ v^0_{int}\ v^1_{int}\ v^2_{int}\ v^3_{int} \end{array} \right\}$$

$\longrightarrow$

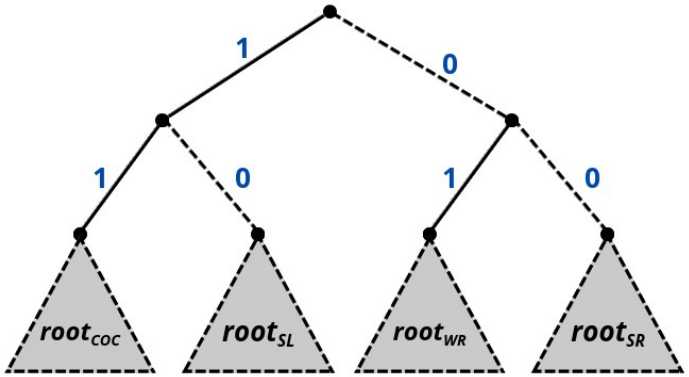


Compression via BDDs

- **Variable ordering :** CUDD reordering function ⟶ BDD size : **46% of reduction** (LUT – aprev = SR)

$$\left[\begin{array}{l}\rho^0, \rho^1, \psi^1, \psi^0, \theta^0, \theta^1, v^1_{int}, v^0_{int}, v^0_{own}, v^1_{own}, \theta^2, \psi^2, \psi^3, \theta^3, \theta^4, \psi^4, \theta^5, \psi^5, \\ v^2_{int}, v^2_{own}, v^3_{int}, v^3_{own}, \rho^4, \rho^5, \rho^3, \tau^3, \rho^2, \tau^2, \tau^0, \tau^1\end{array}\right]$$

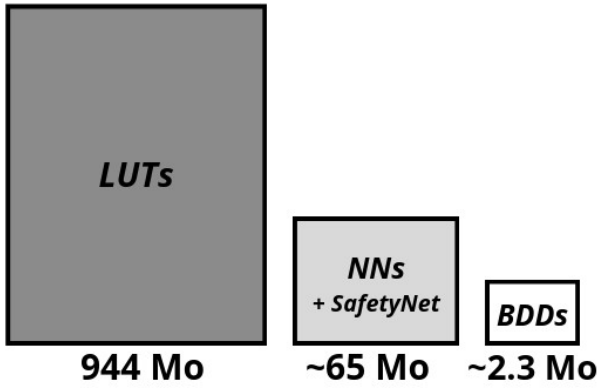
- **A global BDD** is built per LUT by combining the BDDs of each advisory as subgraphs .
- Since advisories are **disjoint** : only 4 subgraphs are needed.



- **$root_A$** : A subgraph indicating whether a given input state results in a A action.

Advantages of BDD-Based Compression

- **Memory footprint reduction:**
  Significant decrease in memory usage.



Advantages of BDD-Based Compression

- **Memory footprint reduction:**
  Significant decrease in memory usage.

- **Improved execution time and deployability:**
  Faster runtime and better suitability for embedded systems.

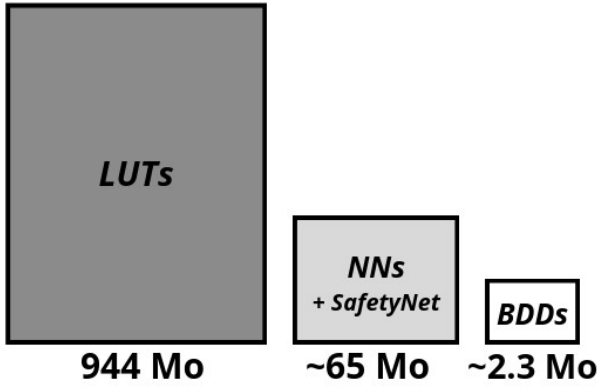


Table 1: Execution time on desktop (μs)

	t_{min}	t_{max}	t_{moy}
LUT	0.970	7963.917	123.348
NN	0.009	3010.368	1.584
BDD	0.087	2640.453	0.453

Table 2: RISC-V (μs)

	t_{min}	t_{max}	t_{moy}
NN	0.500	4574.000	278.890
BDD	3.000	404.250	5.396

Table 3: KRIA KR260 (μs)

	t_{min}	t_{max}	t_{moy}
NN	0.090	1400.424	198.899
BDD	1.690	188.032	3.136

Table 4: ZCU 102 FPGA (μs)

	t_{min}	t_{max}	t_{moy}
BDD	1.200	2.180	1.408

Verifying Domain-Specific Properties with BDDs

Both the system and the properties are encoded as BDDs : **BDD_{acas} / BDD_{prop}**

- **From a property to a BDD**
  - Start from constraints on the state variables (e.g. distance, angles, speeds).
  - Discretize the intervals into valid indices.
  - Encode these indices as Boolean variables.
  - Build a BDD representing all states satisfying the constraints : **BDD_{prop}**

Property 3. If the intruder is directly ahead and is moving towards the ownship, the advisory will not be COC.
Input constraints : $1500.0 \leq \rho \leq 1800.0$, $\psi \geq 3.10$, $v_{own} \geq 980.0$, $v_{int} \geq 960.0$
Desired Output : not COC

- **Property checking**
  - Valid if : **$BDD_{acas} \wedge BDD_{prop} \wedge \neg Output = False$**
  - Otherwise : Counterexample.

Verifying Domain-Specific Properties with BDDs

- **Verification Results**

  Each property is verified on the full ACAS-Xu decision logic using BDDs.

P	BDD_{COC}	BDD_{WL}	BDD_{WR}	BDD_{SL}	BDD_{SR}
P_1	Invalid	Valid	Valid	Invalid	Invalid
P_2	Invalid	Valid	Valid	Invalid	Invalid
P_3	Valid	Valid	Valid	Valid	Valid
P_4	Valid	Valid	Valid	Valid	Valid
P_5	Valid	Valid	Invalid	Valid	Invalid
P_6	Valid	Valid	Valid	Valid	Valid
P_7	Valid	Invalid	Invalid	Valid	Valid
P_8	N/A	Valid	N/A	N/A	N/A
P_9	Valid	Valid	Valid	Valid	Valid
P_{10}	Valid	Valid	Valid	Valid	Valid

Valid : property holds for all states

Invalid : counterexample found

REF : Katz, G., Barrett, C., Dill, D.L., Julian, K., Kochenderfer, M.J.: Reluplex: An efficient SMT solver for verifying deep neural network

BDD as a Verification Tool

- **BDDs are an exact representation of the original LUTs :**
  - Same decision for every state. → Counterexamples are **real behaviors of the LUTs.**

- **Consequence**

  BDDs enable verification of the original LUTs correctness.

- **Expressiveness:**
  - BDDs support more expressive properties

    Can encode **relational properties** between variables

Property 11. If the intruder is approaching the ownship from behind at a lower speed, the network will advise COC.
Input constraints : $v_{int} \leq v_{own}$, $8500.0 \leq \rho \leq 62000.0$, $-3.1416 \leq \theta \leq -3.1416 + 0.01$, $-0.06 \leq \psi \leq 0.06$
Desired Output : COC

Conclusion & Future work

- **From NNs to BDDs**
  - BDDs provide a compact and exact representation of ACAS-Xu decision logic.
  - They enable efficient and predictable execution on embedded platforms.
  - They support formal verification at scale, with real counterexamples.

  - **BDDs are not only a compression technique, but also a powerful verification tool**

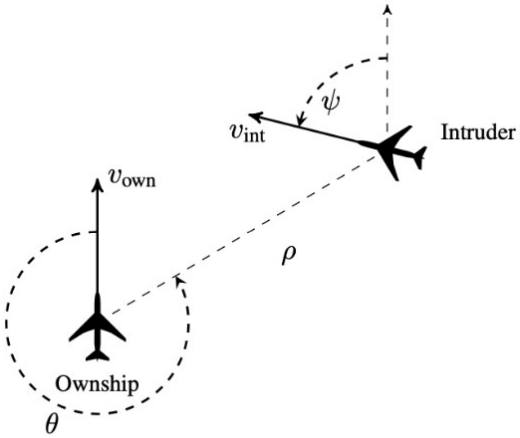
- **Main limitation :**
  - No score encoded so no support for multi-intruder scenarios.
  - It is more complex – ACAS-Xu behavior goes beyond LUTs.

Thank you for listening!
Question ?

The Look Up Tables

Big array of 11 columns (& ~95M lines).

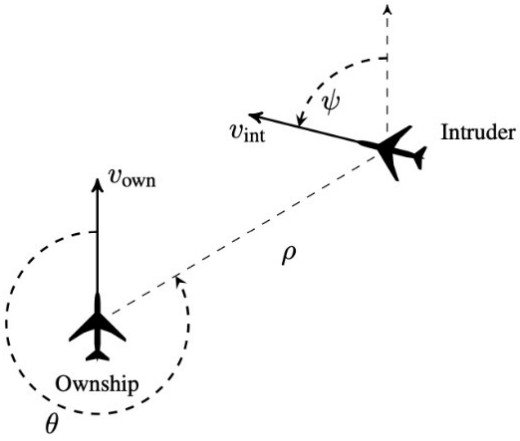
- 6 for the flight and geometric variables.
- 5 for the costs of each possible action (COC, SL, SR, WL, WR)



τ	θ	ψ	V_own	V_int	ρ	Cost_COC	Cost_SL	Cost_SR	Cost_WL	Cost_WR
0.0	-3.14	-3.14	50.0	100	499	24017	24017	24081	24001	24065
32.2	-3.14	0.0	50.0	142	3024	22913	22912	23040	22881	23008
41.7	0.0	3.14	87.6	807	45202	22645	22644	22772	22613	22741
101	3.14	1.5	87.6	1069	763118	20844	20832	21340	20700	21208

The Action Selection

- **Vertical Action :** Multi-Linear Interpolation & Minimal Cost
- **Horizontal Action :** Nearest Neighbor approach & **Lookahead***



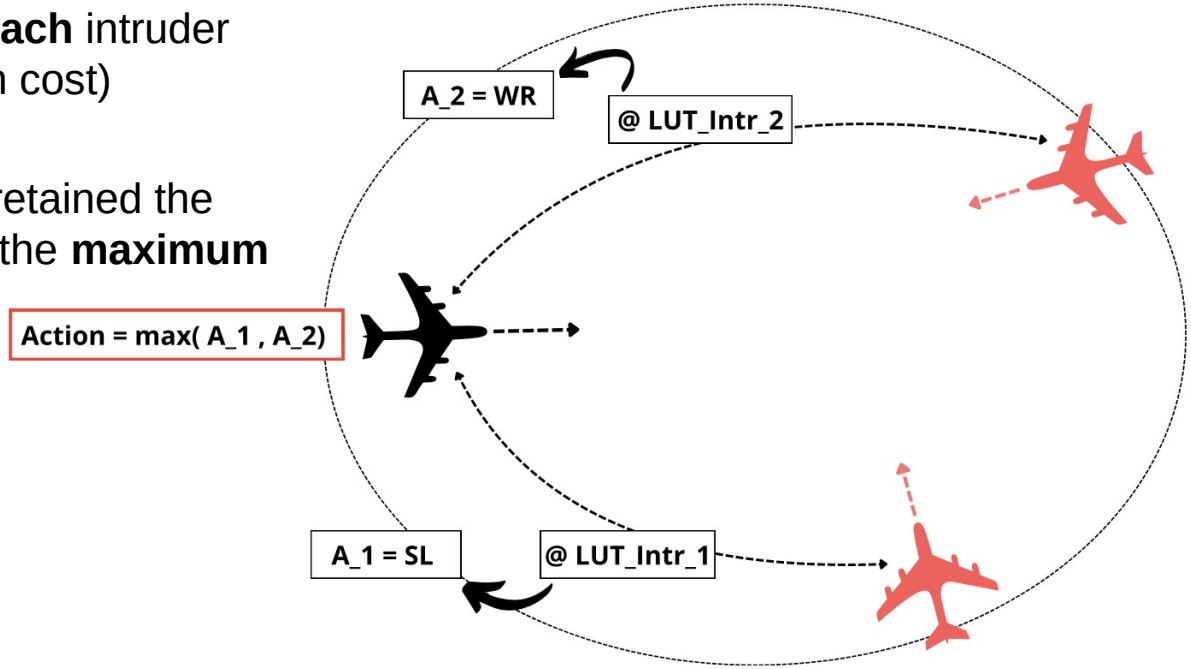
τ	θ	ψ	V_own	V_int	ρ	Cost_COC	Cost_SL	Cost_SR	Cost_WL	Cost_WR
0.0	-3.14	-3.14	50.0	100	499	24017	24017	24081	24001	24065
32.2	-3.14	0.0	50.0	142	3024	22913	22912	23040	22881	23008
41.7	0.0	3.14	87.6	807	45202	22645	22644	22772	22613	22741
101	3.14	1.5	87.6	1069	763118	20844	20832	21340	20700	21208

* : Complex mechanism whithout enouh info, futur work ?

The Action Selection : Multi-Intruder

- Call the tables to determine the avoidance **action** to be performed for **each** intruder (the action with the minimum cost)

- Then among all the actions retained the system selects the one with the **maximum** cost.



Approximations and Hypotheses

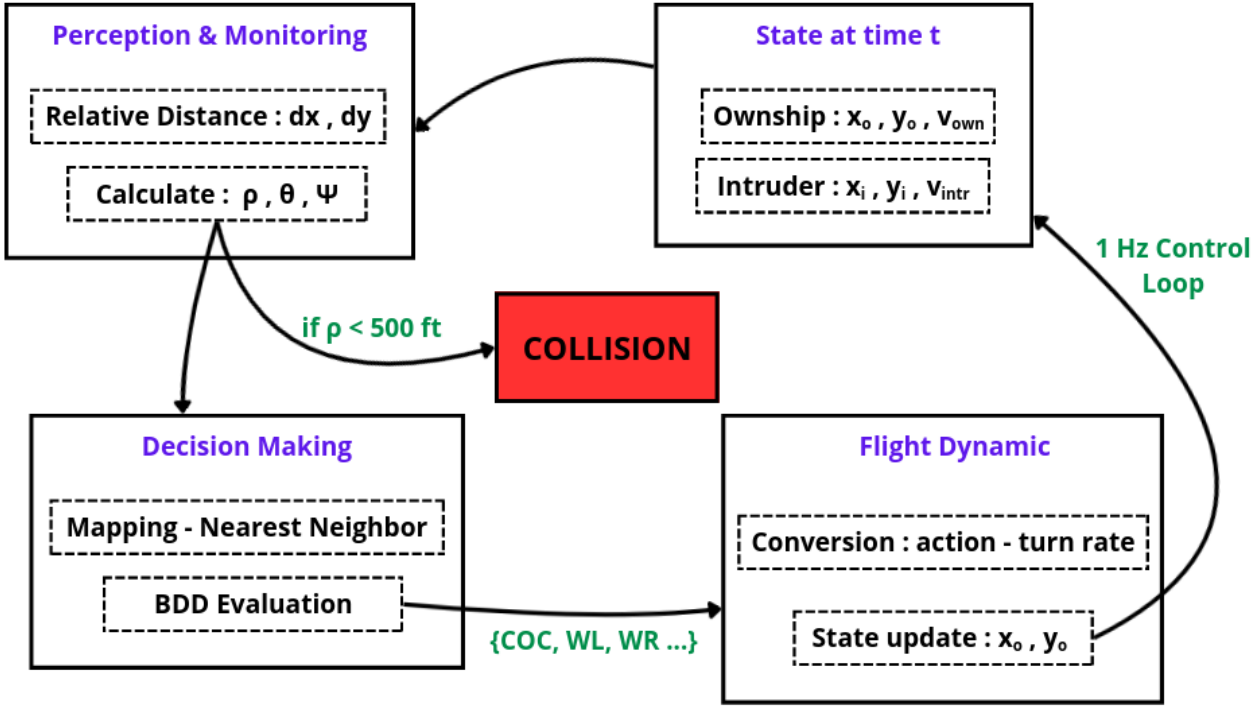
- We consider **Horizontal plan** problems only.
- The ownship and the intruders are in a **non-cooperative** flight. There is no communication about their actions.
- We do not consider the **“lookahead”** mechanism for now.

Closed Loop Verification

- **Closed loop simulation**

  Hypothesis :

  - $\tau = 0$
  - $\mathbf{v}_{own}$, $\mathbf{v}_{intr}$ = constant
  - Intruder going straight

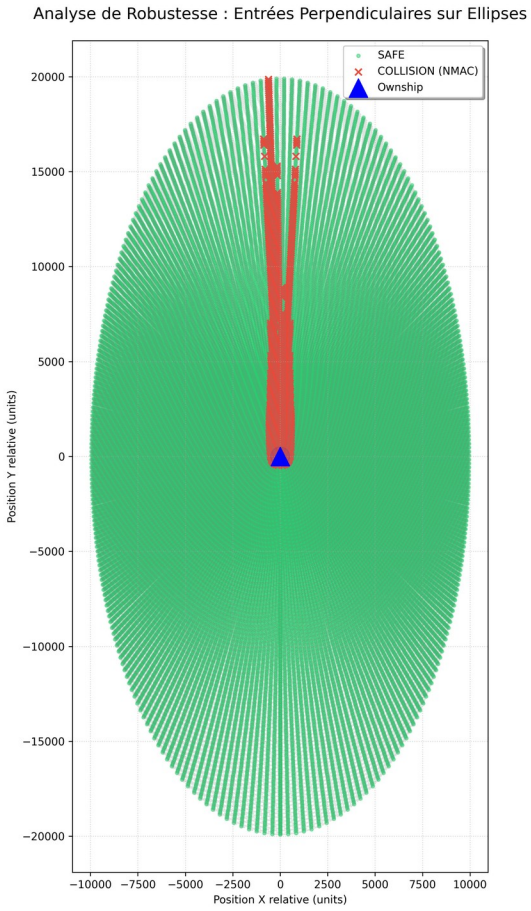


Closed Loop Verification

- **Closed loop simulation**

  Stress-testing the collision avoidance logic through a 360° perpendicular entry sweep on elliptical perimeters.

  x : collision reached during simulation



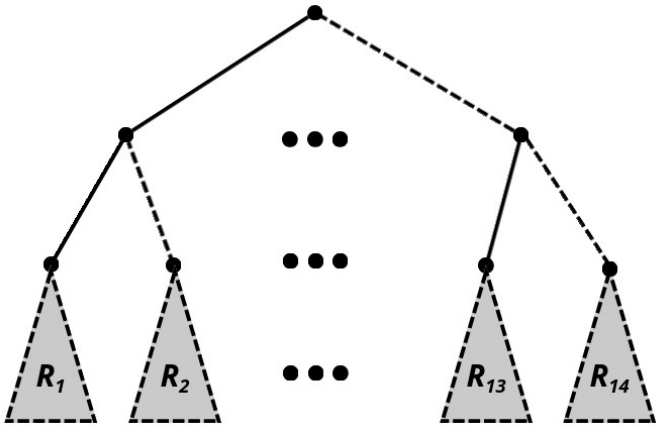
Multi-Intruder using BDDs

- **Encoding the Score from the LUTs**

  Only **score min** are needed : 12203 **min** distinct.

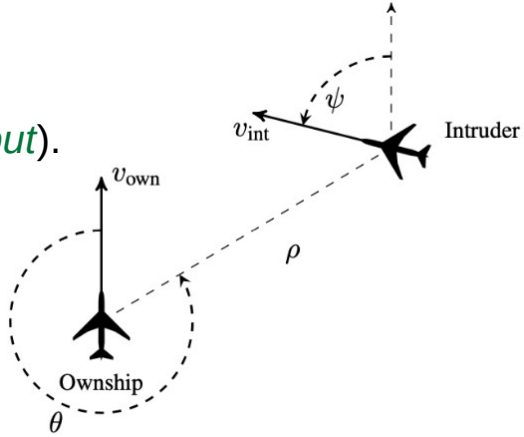
  12203 can be encode with **14 bits**.

  14 subtree classifiers, one for each bits.
  - True = 1
  - False = 0

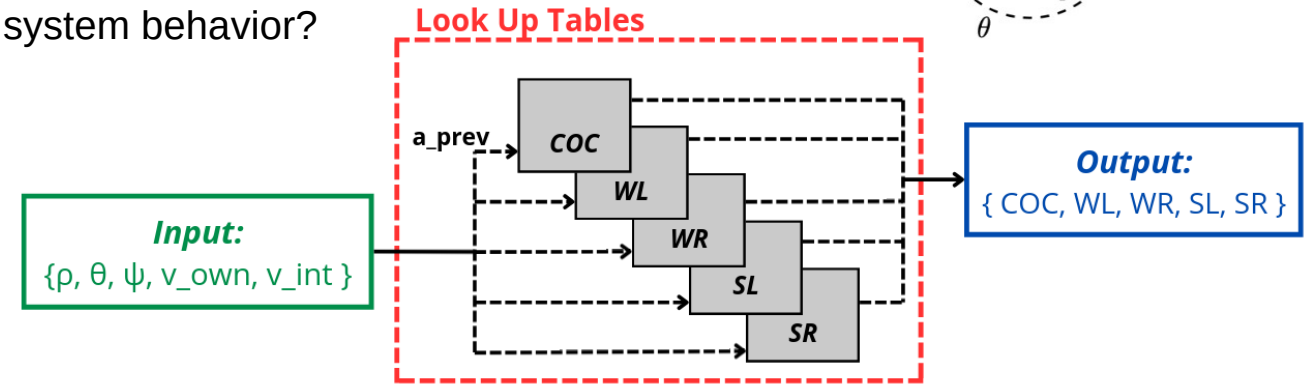


Contexte & Introduction

- **ACAS-Xu** :
  - An airborne collision avoidance system that provides collision avoidance maneuvers (*Output*) based on the current flight state (*Input*).
  - Operates using five **LARGE** lookup tables (LUTs), each 944 MB in size.



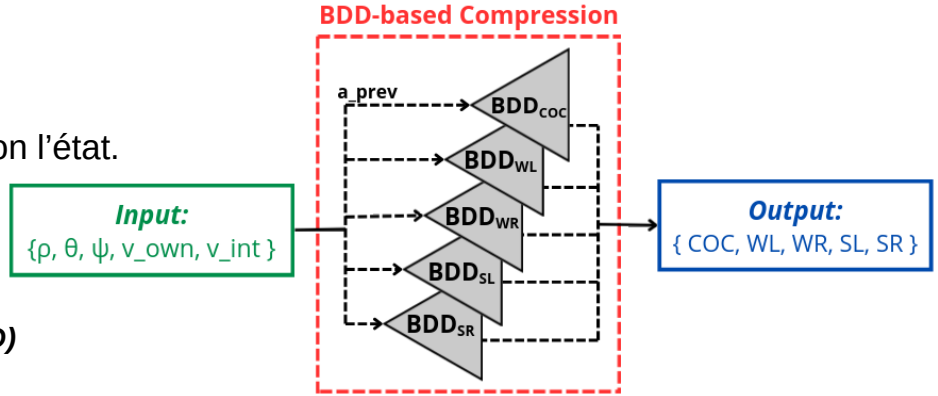
- **Question**: How can the LUT logic be compressed while ensuring correct system behavior?



Compression par Binary Decision Diagrams (BDD)

- **Encodage des données des LUTs dans des BDDs**
  - Un BDD par LUT
  - Classifieur renvoyant *True*/*False* pour chaque action selon l'état.

- **Avantage d'une telle techno :**
  - Gain en empreinte mémoire
    - 944 Mo *(LUT)* > 65 Mo *(NN + Safety Net)* **> ~2.3 Mo *(BDD)***



- Meilleur temps d'execution et embarquabilité : testé sur Desktop, CPU embarqués et FPGA.

Table 1: Execution time on desktop (μs)

	t_{min}	t_{max}	t_{moy}
LUT	0.970	7963.917	123.348
NN	0.009	3010.368	1.584
BDD	0.087	2640.453	0.453

Table 2: RISC-V (μs)

	t_{min}	t_{max}	t_{moy}
NN	0.500	4574.000	278.890
BDD	3.000	404.250	5.396

Table 3: KRIA KR260 (μs)

	t_{min}	t_{max}	t_{moy}
NN	0.090	1400.424	198.899
BDD	1.690	188.032	3.136

Table 4: ZCU 102 FPGA (μs)

	t_{min}	t_{max}	t_{moy}
BDD	1.200	2.180	1.408

Neural Network Evaluation

- **Approach A : Verification of the good behavior through Scenarios**

  Identification of weak regions through verification of domain-specific properties (Single AND Multi-intruder), based on 10 properties from the literature and 7 additional.

Verification tools on CAISAR:

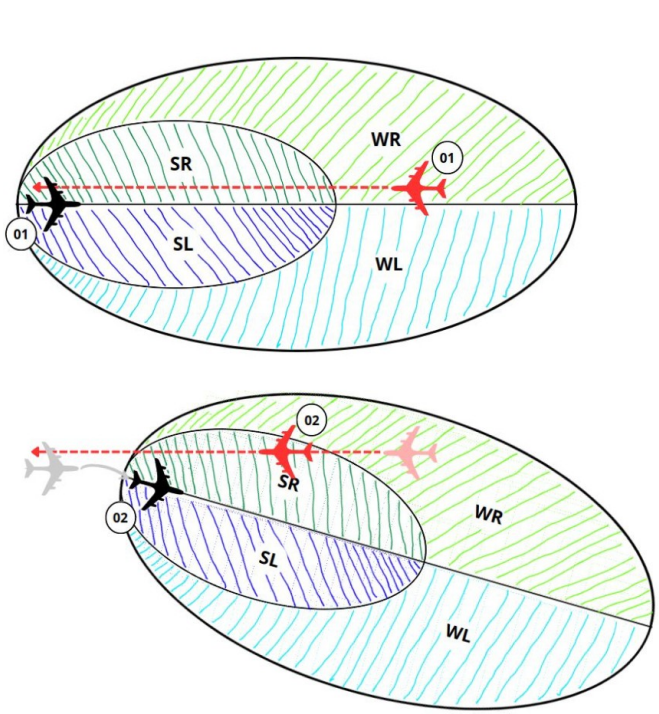
- Marabou
- α - β -CROWN

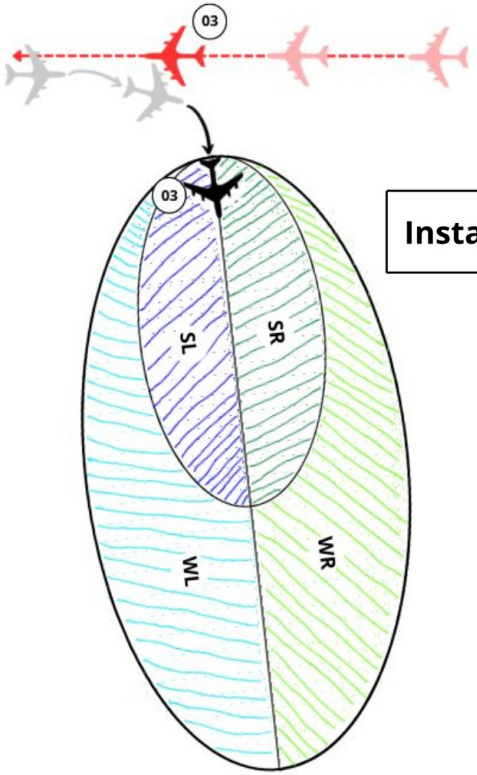
PROPERTY 2. *If the intruder is approaching the ownship from behind at a lower speed, the network will advise COC.*

Input constraints : $55000 \leq \rho \leq 10000$, $-3.1416 \leq \theta \leq -3.1416 + 0.01$, $-0.06 \leq \psi \leq 0.06$, $800 \leq v_{own} \leq 1200$, $0.0 \leq v_{int} \leq 300$

Desired Output : COC

Anexe - The “Lookahead” mechanism





	01	02	03
Instantaneous Action	WR	SR	COC

→ **Final Action = SR**